Reproducing the Dragonblood WPA3/SAE Vulnerabilities in a Containerized Virtual-Radio Laboratory

Vladimir Chukhlebov
University of Padua, Network Security
vladimir.chukhlebov@studenti.unipd.it

Abstract

WPA3 replaced WPA2's Pre-Shared Key exchange with Simultaneous Authentication of Equals (SAE), a password-authenticated key exchange built on the Dragonfly handshake. In 2019 and 2020, Vanhoef and Ronen showed in their Dragonblood work that SAE's password-to-group derivation leaks information through timing and cache side channels, and that WPA3's transition mode can be downgraded to WPA2. This paper documents a self-contained laboratory that reproduces three of those findings: an SAE timing side channel, a WPA3 to WPA2 downgrade, and a full password-recovery chain, all against a deliberately vulnerable `hostap_2_4` access point. The lab runs entirely on virtual Wi-Fi radios (`mac80211_hwsim`) inside Docker containers, with a Lima VM providing the Linux kernel on macOS, so no radio hardware is required. Following a three-phase plan (software mock, emulation, physical hardware), we implement each attack behind a common wireless-backend interface. We validate the attacks with 117 automated tests and confirm the timing leak statistically (Welch's t-test, $p = 0.036$). The recovery step narrows a 100,000-word dictionary to a single candidate using nine spoofed MAC addresses, recovering the target password. The same recovery code was also run against a real and authorized home router; it reported no usable timing signal, which is the expected result for current, patched firmware. We also describe the main practical problem, namely that sub-microsecond SAE computation made the timing difference invisible in emulation, and its solution: a configurable per-iteration delay added to hostapd.

1 Introduction

WPA3-Personal is the current consumer Wi-Fi security standard, and its central improvement over WPA2 is the SAE handshake, which resists the offline dictionary attacks that affect WPA2's four-way handshake. Because SAE is now the default on new hardware, weaknesses in it matter more in practice than attacks on WPA2, which is already known to be broken.

The Dragonblood work of Vanhoef and Ronen[1] showed that early SAE implementations leak password information through side channels and that WPA3's backward-compatibility (transition) mode can be downgraded. This project reproduces those results in a controlled, fully scripted environment, so the attacks can be studied, tested, and re-run without physical Wi-Fi hardware or a live target network.

Contributions:

- A containerized WPA3 attack lab built on virtual radios (`mac80211_hwsim`), runnable with a single `make up` command on Linux or macOS (via a Lima VM), targeting a pinned vulnerable hostapd.
- Three attacks implemented behind one wireless-backend interface: an SAE timing side channel, a WPA3 to WPA2 downgrade, and a multi-MAC timing-to-password recovery chain. The same recovery code runs against the emulated lab and, with the `live-recovery` tool, against a real AP.
- A statistical analysis pipeline (calibration, Welch's t-test, effect size, power analysis) with reproducible CSV, JSON, and figure output.
- A test suite of 117 automated tests across unit, integration, and full Docker end-to-end levels, and a plain account of the engineering problems encountered and how they were solved.

2 Background: Dragonblood

2.1 The SAE (Dragonfly) handshake

SAE[6,7] is a password-authenticated key exchange. Both peers derive a shared secret element, the Password Element (PWE), from the network password and their two MAC addresses, then exchange Commit frames (containing a scalar and an elliptic-curve element) followed by Confirm frames that prove knowledge of the derived key. Unlike WPA2, neither the password nor a directly crackable hash is exposed on the wire, which is what makes SAE resistant to offline dictionary attacks.

2.2 Hash-to-curve and the hunting-and-pecking loop

The vulnerable step is PWE derivation. For elliptic-curve groups, early SAE uses a try-and-increment (hunting-and-pecking) loop: for counter = 1, 2, and so on, it computes a candidate seed, HMAC-SHA256 of (MAC_A, MAC_B) keyed with (password, counter), interprets it as an x-coordinate on the curve, and tests whether $x^3 + ax + b$ is a quadratic residue modulo the field prime p. The loop stops at the first counter that yields a valid point. Importantly, the number of iterations depends on the password and the MAC pair. On the NIST P-256 curve about half of all x values are valid, so the iteration count varies from password to password.[1] We reuse this exact computation later, offline, to filter dictionary candidates.

2.3 The timing side channel

If the loop is not constant-time, the response latency of the AP correlates with the iteration count, which in turn is a function of the password. An attacker who sends SAE Commit frames from many different spoofed MAC addresses observes, for each MAC, an iteration count for the (unknown) password. Vanhoef and Ronen report that with fewer than 75 timing measurements per spoofed MAC address they can recover the number of executed iterations.[2] The recovered per-MAC iteration counts then act as a filter: any candidate password whose computed iteration counts (which the attacker can calculate offline) disagree with the observed counts is eliminated. Intersecting across enough MAC addresses isolates the password.

2.4 Other Dragonblood attack classes

Beyond timing, Dragonblood[1,2] identifies cache-based side channels (memory-access patterns during PWE derivation), transition-mode downgrade (a WPA3/WPA2 network pushed onto WPA2 by a rogue AP), security-group downgrade via forged decline messages, and denial of service, where as few as 16 forged commit frames per second drive high CPU load on an AP. This project implements the timing side channel, its password-recovery extension, and the transition-mode downgrade.

2.5 Disclosure timeline and CVEs

The initial disclosure was in April 2019. In August 2019 the Wi-Fi Alliance's private mitigation (recommending Brainpool curves) was found to introduce a second side channel; updated guidance followed in January 2020, and the full paper appeared at IEEE S&P 2020.[1] Relevant identifiers include CVE-2019-9494 (SAE side channels),[3] CVE-2019-13377 (Brainpool timing),[4] and the EAP-pwd family such as CVE-2019-9497.[5]

3 Threat Model and Scope

The attacker is within radio range of the target AP, can transmit and receive 802.11 management and authentication frames, and can spoof arbitrary source MAC addresses, but does not know the password. The target is a pinned, pre-patch build of hostapd (the `hostap_2_4` release tag, from before the constant-time Dragonblood fixes), configured for WPA3-SAE on NIST P-256 (group 19). This mirrors the deployed embedded devices that motivated the original research. Modern, patched APs use constant-time PWE derivation, so the lab targets the historical vulnerable behavior, not current firmware.

4 Implementation Plan and Phases

The project was structured as a three-phase progression, where each phase reduces uncertainty before adding real-world complexity:

1. **Mock phase.** The attack logic (frame construction, timing collection, calibration, statistics, dictionary filtering) is exercised against an in-process model of a vulnerable SAE responder (`VulnerableMockBackend`), which computes the true iteration count for a password and MAC pair and returns a modeled response time. This checks correctness with no hardware or containers.

2. **Emulation phase.** The same attack code runs unchanged against a real hostapd, over virtual radios provided by the Linux `mac80211_hwsim` kernel module, inside Docker containers.

3. **Physical phase.** Reproduction against real Wi-Fi hardware to confirm the emulation findings, done with the `live-recovery` tool (Section 7).

The key enabler is a single interface, `WirelessBackend`, with concrete implementations for the mock, and for real or emulated interfaces via Scapy. Attack modules depend only on this interface, so moving from mock to emulation to hardware does not change a line of attack logic; only the injected backend changes. The physical phase follows directly from this design: `live-recovery` runs the same recovery attack against a real access point, adding only what a live target needs (monitor-mode setup, channel discovery, and a verdict), rather than reimplementing anything.

5 System Architecture

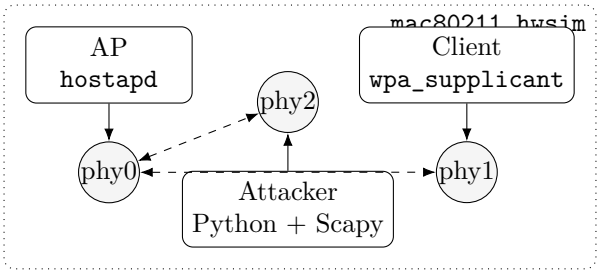


Figure 1: Lab topology. Three containers (AP, Client, Attacker) each own a virtual radio created by `mac80211_hwsim`; frames travel through the kernel rather than the air. On macOS a Lima VM supplies the Linux kernel.

5.1 Containers and virtual radios

The lab (Figure 1) has three Docker containers: an **AP** running the vulnerable hostapd, a **Client** running `wpa_supplicant` (used to establish a legitimate SAE baseline), and an **Attacker** running the Python and Scapy attack tools. The AP and Client use `network_mode: none`; the Attacker uses host networking for notebook access. All run privileged with `pid: host` so they can manage wireless interfaces. The `mac80211_hwsim` module[8] creates three virtual PHYs on the host; `host/setup.sh` loads the module and `host/assign-phys.sh` moves each PHY into a container's network namespace. Because frames pass through the kernel instead of the air, the channel is noise free and deterministic. As Section 8 explains, that property also removed the timing difference the attacks rely on.

5.2 macOS support via Lima

`mac80211_hwsim` is a Linux kernel module and is not available on macOS. A Lima VM (`lima/wifi-lab.yaml`) provides an Ubuntu guest with Docker and the wireless stack; the `Makefile` detects the OS and forwards every target into the VM via `host/vm-exec.sh`, so the developer experience is the same on both platforms.

5.3 Image hierarchy and configuration

A four-layer Docker build keeps images small and cache friendly: `wifi-lab-base`, then `wifi-lab-builder` (hostap source and build dependencies), then the AP and Client (compile plus runtime). The Attacker image extends the base directly. Attack-specific infrastructure changes, such as putting the AP into transition mode for the downgrade experiment, are layered as Compose overlays (`docker-compose.downgrade.yml`) rather than duplicated images. Configuration files (`hostapd_sae.conf`, its transition variant, and `wpa_supplicant_sae.conf`) are bind-mounted, not baked in.

5.4 Attack framework

Every attack shares one lifecycle: reconnaissance, execution, then analysis. Reconnaissance scans for the target AP; execution collects the raw timing or capture data; analysis runs the statistics and writes the report. Configuration is passed as typed, immutable objects rather than loose dictionaries. Each run writes a timestamped directory under `results/` containing the raw samples (CSV), the analysis report (JSON), run metadata, and plots, so every result in this paper is reproducible from committed artifacts.

6 Implementation Walkthrough

Bringing up the lab and running an attack is a short sequence:

1. `make up`: on macOS, start the Lima VM, load `mac80211_hwsim`, build the images, launch the three containers, and assign the virtual PHYs to their namespaces.
2. The AP loads `hostapd_sae.conf` (SSID `DragonbloodLab`, `wpa_key_mgmt=SAE`, PMF required, channel 6); the Client associates over SAE to confirm a working WPA3 network.
3. `make attack-timing`, `attack-downgrade`, or `attack-recovery` resolves the AP's BSSID and runs the matching CLI entry point inside the Attacker container, writing artifacts to `results/`.

The three CLI entry points (`timing-attack`, `downgrade-attack`, `recovery-attack`) are declared in `pyproject.toml`. Each accepts the shared arguments `--interface`, `--bssid`, `--ssid`, `--channel`, and `--output-dir`.

7 Attacks and Results

All results below are the reference runs committed under `results/` (netem profile: clean).

7.1 SAE timing side channel

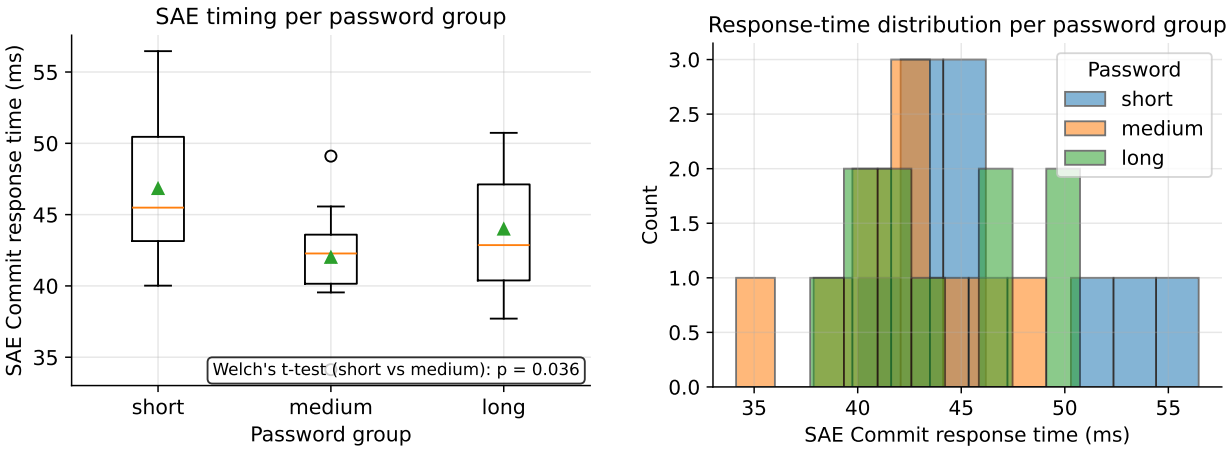
The timing side-channel attack measures the AP's SAE Commit response time for several password groups. For each group it repeatedly resets AP state with a deauthentication, sends a freshly crafted SAE Commit frame, and records the round-trip latency at nanosecond resolution. It then computes per-group descriptive statistics and runs pairwise Welch's t-tests (unequal variance, $\alpha = 0.05$).

Table 1 reports the reference run (ten samples per group). The mean response time is highest for the short group (46.8 ms) and lower for medium (42.0 ms) and long (44.0 ms). The short-versus-medium difference is statistically significant ($p = 0.036$); the other pairs are not, at this small sample size. Figures 2a and 2b show the distributions. The significant pair confirms that the emulated AP's SAE timing is password-dependent, so the side channel is present and detectable.

The absolute latencies here reflect the per-iteration delay configured for this run; only the relative differences between groups matter.

Table 1: SAE timing side channel: per-group statistics and pairwise Welch's t-tests (10 samples per group).

Password group	Mean (ms)	Median (ms)	Std (ms)
short	46.8	45.5	5.40
medium	42.0	42.3	3.97
long	44.0	42.9	4.55
Comparison	**p-value**		**Significant**
short vs medium	0.036		yes
short vs long	0.220		no
medium vs long	0.314		no



(a) Response-time distribution per group.

(b) Overlaid response-time histograms.

Figure 2: SAE Commit response times per password group. The short group is slower, consistent with a larger hash-to-curve iteration count.

7.2 Password recovery via multi-MAC intersection

The password-recovery attack turns the timing leak into an actual password. It collects SAE Commit timings from up to 15 spoofed source MAC addresses, 200 samples each in the reference run, then does two things:

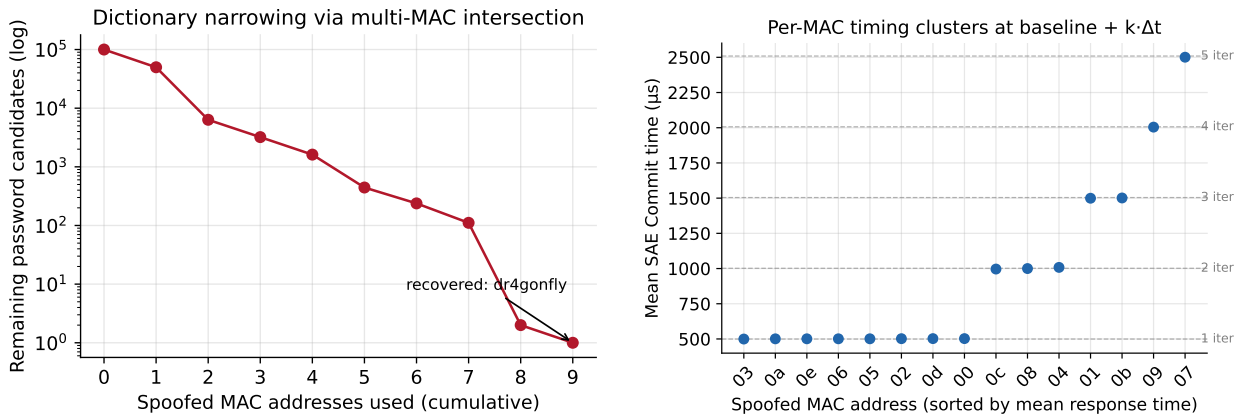
1. **Calibrates.** The calibration step takes the mean response time per MAC, sets the baseline to the smallest mean, and finds the per-iteration time Δt as the smallest gap between sorted means that exceeds a noise threshold of three times the average standard error. The estimated iteration count for a MAC is $\text{round}((\text{mean} - \text{baseline})/\Delta t) + 1$. The reference run calibrated to a baseline of about 499 μs, a Δt of about 491 μs, and six distinct clusters (Figure 3b).

2. **Filters.** Starting from the full 100,000-word dictionary, it computes each candidate's true iteration count for each MAC offline and keeps only candidates whose computed count matches the observed one. Intersecting across MACs shrinks the candidate set monotonically.

Table 2 and Figure 3a show the narrowing: after nine MAC addresses the 100,000-word dictionary collapses to a single candidate, and the attack recovers the target password (`dr4gonfly`). This is the main result: a correct password, embedded in a large dictionary, extracted purely from response-time statistics.

Table 2: Password recovery: per-MAC estimated iteration count and the remaining candidate set after intersecting each MAC.

MAC (last byte)	Est. iterations	Candidates remaining
:00	1	49,853
:01	3	6,341
:02	1	3,212
:03	1	1,615
:04	2	444
:05	1	238
:06	1	111
:07	5	2
:08	2	**1**



(a) Candidate narrowing (log scale).

(b) Per-MAC mean time, by iteration count.

Figure 3: Password recovery. (a) Nine spoofed MACs reduce 100,000 candidates to one. (b) Per-MAC means cluster at baseline $+ k\,\Delta t$ for integer k, the structure the calibrator uses.

7.3 WPA3 to WPA2 downgrade

The downgrade attack targets WPA3 transition mode, in which an AP advertises both SAE and WPA2-PSK for backward compatibility. The attack forges beacons for the same SSID whose RSN Information Element advertises only WPA2-PSK, built by stripping the SAE AKM suite (00-0F-AC:8) and leaving PSK (00-0F-AC:2). A background sniffer watches for a resulting WPA2 four-way handshake, which would indicate the client downgraded and can then be attacked with classic offline WPA2 techniques.

In the reference run the attacker injected 71 forged beacons over five seconds and captured zero WPA2 handshakes: the emulated client did not downgrade. We report this outcome plainly. It reflects the emulated client's configuration (the Client is set to require SAE, so it correctly refuses

WPA2), not a flaw in the forging logic, which unit tests confirm produces a well-formed WPA2-only RSN IE. The mechanism and its success condition (a captured handshake) are the contribution here; chaining a successful downgrade into a full WPA2 compromise (for example KRACK or a dictionary attack on the captured handshake) is left as future work.

7.4 Physical phase: field test against a real router

To move from emulation to hardware we built `live-recovery`, a command-line tool that runs the same recovery attack against a real access point. Over the emulated version it adds only what a live target requires: enabling monitor mode, sweeping channels to discover the AP, and mapping the result to a plain verdict, one of `vulnerable-recovered`, `vulnerable-partial`, `patched-no-signal`, or `target-not-found`. The "no signal" case is detected precisely: calibration reports no signal when every per-MAC mean falls within the noise floor, which is the signature of a constant-time PWE derivation.

We ran it against the author's own and authorized home router, a WPA2/WPA3 transition-mode access point. The calibrator found no per-iteration timing signal above noise, so the tool reported `patched-no-signal`: no usable side channel. This is the expected and correct outcome for current firmware, whose SAE derivation is constant-time. The negative result is useful on two counts: it confirms that the deployed Dragonblood mitigation is effective, and it shows that the calibration step does not report a signal that is not there (no false positive). A positive field result would require a known-vulnerable legacy device.

8 Problems Encountered and Solutions

8.1 Sub-microsecond SAE made timing invisible

The main problem: in emulation, `mac80211_hwsim` passes frames through the kernel with no radio latency, and hostapd's PWE derivation on a modern CPU completes in well under a μs per iteration. The per-iteration timing signal was therefore far below measurement noise, so the effect the attack relies on could not be measured.

The solution was to make the iteration cost explicit and adjustable by patching hostapd to sleep a configurable interval after each hash-to-curve iteration (Listing 1). The delay is read once from the `SAE_ITERATION_DELAY_US` environment variable (default 3000 μs, that is 3 ms) and applied inside hostapd's PWE-derivation loop. This models the slower PWE derivation of the embedded devices Dragonblood targeted, and it changes only the size of the per-iteration cost, not the attack logic, which is identical to what would run against real hardware. We set the default to 3 ms from the emulation's own jitter: the timing run's standard deviation is about 4.5 ms, so with 200 samples per MAC the calibration threshold is about 1 ms. A 3 ms delay therefore clears the noise floor with margin, while 1 ms would sit on it.

Listing 1: The per-iteration timing delay added to hostapd's SAE PWE derivation (`timing_delay.patch`).

```
@@ src/common/sae.c: sae_derive_pwe_ecc()
   if (hmac_sha256_vector(addrs, sizeof(addrs), 2,
                          addr, len, pwd_seed) < 0)
       break;
+ {
+ static int delay_us = -1;
```

```
+ if (delay_us < 0) {
+ const char *env =
+ getenv("SAE_ITERATION_DELAY_US");
+ delay_us = env ? atoi(env) : 3000;
+ }
+ usleep(delay_us);
+ }
   res = sae_test_pwd_seed_ecc(sae, pwd_seed, ...);
```

8.2 Noise and discrete iteration counts

Even with an amplified signal, individual measurements are noisy. Rather than trusting single samples, the calibrator works on per-MAC means and uses the fact that true iteration counts are integers: means cluster at baseline $+ k\,\Delta t$ for integer k (Figure 3b). A 3σ-style noise threshold separates genuine gaps between clusters from jitter, and a validation step warns if larger gaps are not clean integer multiples of Δt.

8.3 macOS has no `mac80211_hwsim`

As noted in Section 5, this was solved with a Lima VM and Makefile forwarding, so the lab is a single command on both Linux and macOS.

8.4 Reproducibility versus realism

For deterministic tests the password is fixed (`dr4gonfly`) across the AP config and the recovery assertions. This ties tests to a known value, a deliberate trade-off that favors reproducible, checkable end-to-end runs over realism; the attack itself makes no use of the known password.

8.5 Physical phase

For the physical phase we built `live-recovery` (Section 7.4), which runs the same recovery attack against a real access point, adding monitor-mode setup, channel discovery, and a verdict. Run against the author's own patched home router, it recovered nothing and reported `patched-no-signal`, the expected result, since current firmware derives the password element in constant time. This is not a failure: it confirms the deployed fixes work and that the tool does not produce false positives. One caveat surfaced during this work: the default channel sweep covers 2.4 GHz (channels 1, 6, 11), so 5 GHz targets such as the test router require an explicit `--channel` (or `--channels`).

9 Testing and Validation

The lab is covered by 117 automated tests (Table 3), organized as a pyramid. Unit tests (75) cover pure logic: SAE and RSN frame construction, the statistics and calibration math, plotting, and channel discovery. Integration tests (20) drive full attack pipelines against the mock backend, including the end-to-end recovery of `dr4gonfly` from a small dictionary and the `live-recovery` tool (vulnerable, patched, and target-not-found verdicts). End-to-end tests (22) stand up the actual Docker lab over `mac80211_hwsim` and check real SAE association, attack exit codes, and the presence and shape of the CSV, JSON, and plot artifacts.

Table 3: Automated test suite (117 tests total).

Level	Scope	Count
Unit	frame/RSN/crypto, stats, calibration, discovery	75
Integration	mock-backend and live-recovery pipelines	20
End-to-end	full Docker lab over `mac80211_hwsim`	22
Total		**117**

10 Limitations and Future Work

The lab targets NIST P-256 (group 19) only; Brainpool and MODP groups, which show their own timing behavior,[4] are not yet modeled. The timing signal is amplified by an injected delay; while the attack logic is identical to a hardware attack, absolute latencies are not representative of a specific device. Physical validation against a known-vulnerable device is still open: the only accessible real AP was patched (the expected modern case), so the field test correctly returned no signal. A natural next step is to buy several of the cheapest popular WPA3 routers on Amazon or Alibaba, which often ship outdated firmware, and test them for the timing side channel. Also open are completing the downgrade-to-WPA2 attack chain (capturing and cracking a real four-way handshake, or chaining KRACK) and a 5 GHz-aware default channel sweep. The design is extensible, so FragAttacks and cache side channels are natural next targets.

11 Conclusion

We reproduced three Dragonblood results, an SAE timing side channel, a transition-mode downgrade, and a full timing-to-password recovery, in a scripted, hardware-free lab built on virtual radios and a pinned vulnerable hostapd. The recovery step extracted a correct password from a 100,000-word dictionary using nine spoofed MAC addresses, and the timing leak was confirmed statistically. The main lesson is that emulation's determinism removes the very timing signal these attacks rely on, and that a small, configurable delay in PWE derivation restores an observable and faithful side channel without changing the attack. The result is a reproducible platform for studying WPA3's weaknesses and, by extension, the value of the constant-time fixes now deployed.

A Reproducibility

Prerequisites. Docker and `make`. On macOS, Lima (`brew install lima`); the `Makefile` detects the OS and runs the Linux steps inside the VM.

Bring up the lab.

```
make up # VM (macOS) + load mac80211_hwsim +
          # build images + start AP/Client/Attacker
make test # verify SAE association (AP and Client)
```

Run the attacks. Artifacts are written to `results/<attack>/<timestamp>/` (`raw_samples.csv`, `analysis.json`, `metadata.json`, `plots/`):

```
make attack-timing # SAE timing side channel
make attack-downgrade # WPA3 to WPA2 downgrade
make attack-recovery # timing to password recovery
```

Physical (live) target. Against a real and authorized AP, on a Linux host with a monitor-capable adapter:

```
sudo make attack-live IFACE=wlan0 SSID='HomeWiFi' \
  DICT=/path/wordlist.txt
# 5 GHz targets: add --channel <n> (default sweep is 2.4 GHz)
```

The verdict is one of vulnerable-recovered, vulnerable-partial, patched-no-signal, or target-not-found.

Tuning and hardware-free runs. The timing amplitude is set by `SAE_ITERATION_DELAY_US` (default 3000). Any attack accepts `--dry-run` to use the in-process modeling backend, so no lab is required.

Tests.

```
make test-unit # unit + integration (local, no VM)
make test-e2e # brings up the lab, runs all e2e tests
```

Rebuilding this paper. Run `docs/paper/build.sh`, which regenerates the figures from `results/` and compiles `paper.tex`.

References

[1] M. Vanhoef and E. Ronen, "Dragonblood: Analyzing the Dragonfly handshake of WPA3 and EAP-pwd," in *2020 IEEE Symp. on Security and Privacy (S&P)*, 2020, pp. 517-533.

[2] M. Vanhoef and E. Ronen, "Dragonblood: Analysing WPA3's Dragonfly handshake," 2019. Available: `https://wpa3.mathyvanhoef.com/`

[3] MITRE, "CVE-2019-9494: SAE side-channel attacks (Dragonblood)," 2019. Available: `https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-9494`

[4] MITRE, "CVE-2019-13377: Timing side-channel in WPA3 SAE with Brainpool curves," 2019. Available: `https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-13377`

[5] MITRE, "CVE-2019-9497: EAP-pwd reflection and side-channel attacks," 2019. Available: `https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-9497`

[6] IEEE, *IEEE Std 802.11-2016*, Sec. 12.4 (Authentication using a password, SAE), 2016.

[7] D. Harkins, "Dragonfly key exchange," RFC 7664, IETF, Nov. 2015.

[8] Linux Kernel Documentation, "mac80211_hwsim: software simulator of 802.11 radios." Available: `https://www.kernel.org/doc/html/latest/networking/mac80211_hwsim/mac80211_hwsim.html`